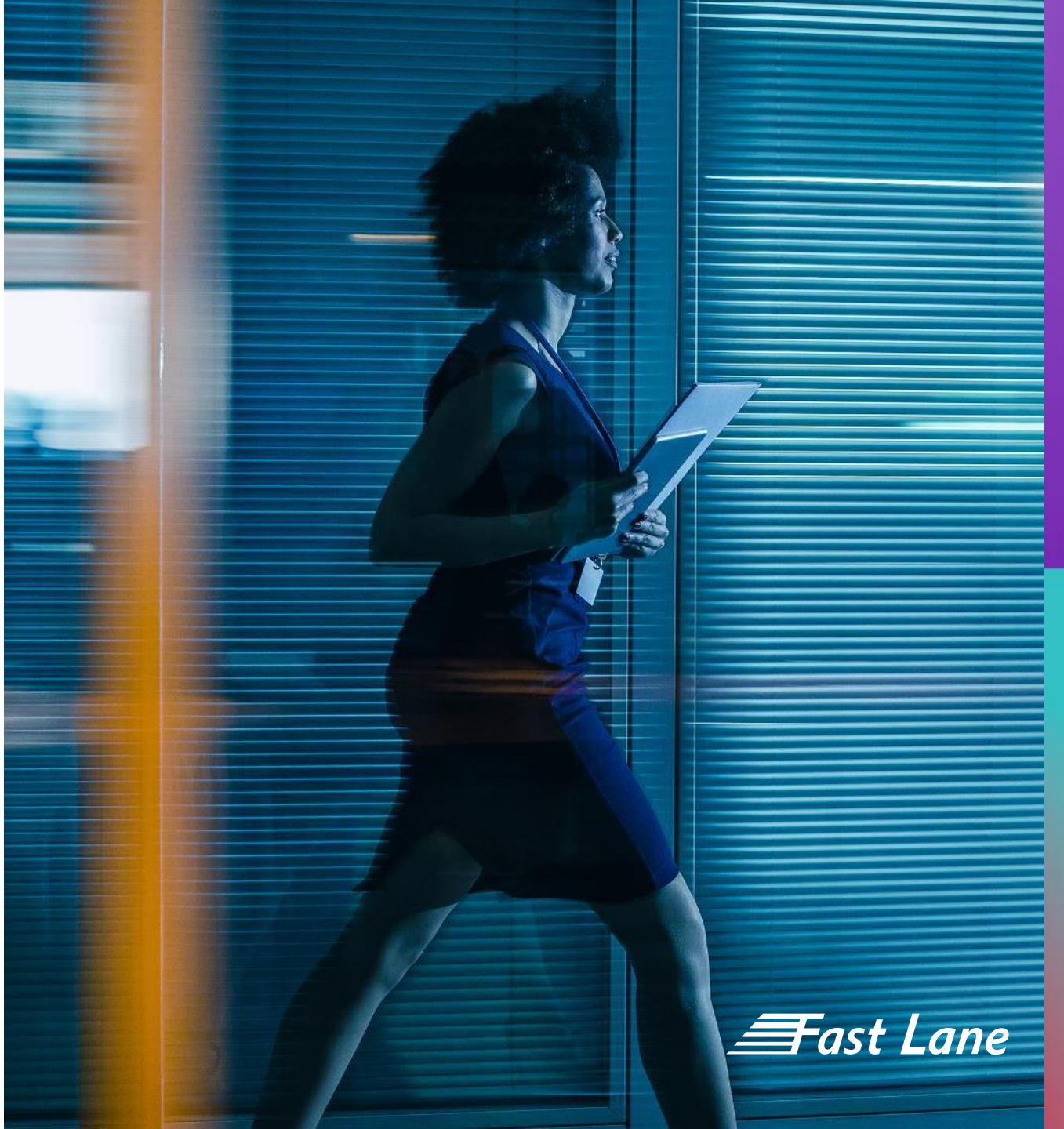
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 4

Extending Copilot Studio Agents with Tools

From answering questions to calling real systems

Automate Business Processes with Copilot Studio and Copilot Cowork



01

Prompt Tool & Code Interpreter

A fixed instruction for the canteen order

02

Dataverse: Knowledge & Business Records

Travel claims saved as table rows

03

Connector Tools: Prebuilt Connectors

One Teams action, filled from the chat

04

Agent Flows: Deterministic Automation

Fixed steps with a prompt and a reviewer

05

Custom Connectors & REST APIs

Live canteen stock from an OpenAPI file

06

MCP Servers as Agent Tools

Staff and shifts discovered at runtime

07

Computer Use: Apps Without APIs

Reading and filling web apps by screen

Tools in Copilot Studio

From answering questions to calling real systems

- A grounded agent answers questions; a **tooled agent acts** on the systems a team already uses
- Seven tool types: prompts, Dataverse, connectors, agent flows, REST APIs, MCP servers and **computer use**
- Each demo gives a CS- agent one real job, starting with the **canteen order**



Overview

Seven tool types, one demo agent each

- A **prompt** calculates the canteen order, and a Dataverse table keeps every travel claim
- One Teams action posts a trip summary, and two **agent flows** run the Health Advisor's fixed steps
- Live canteen stock comes from a **REST API**, staff and shifts from an MCP server
- **Computer use** reads and fills web apps that have no API
- Each tool type has a job it does best, and choosing the right one is the maker skill



01

Prompt Tool & Code Interpreter

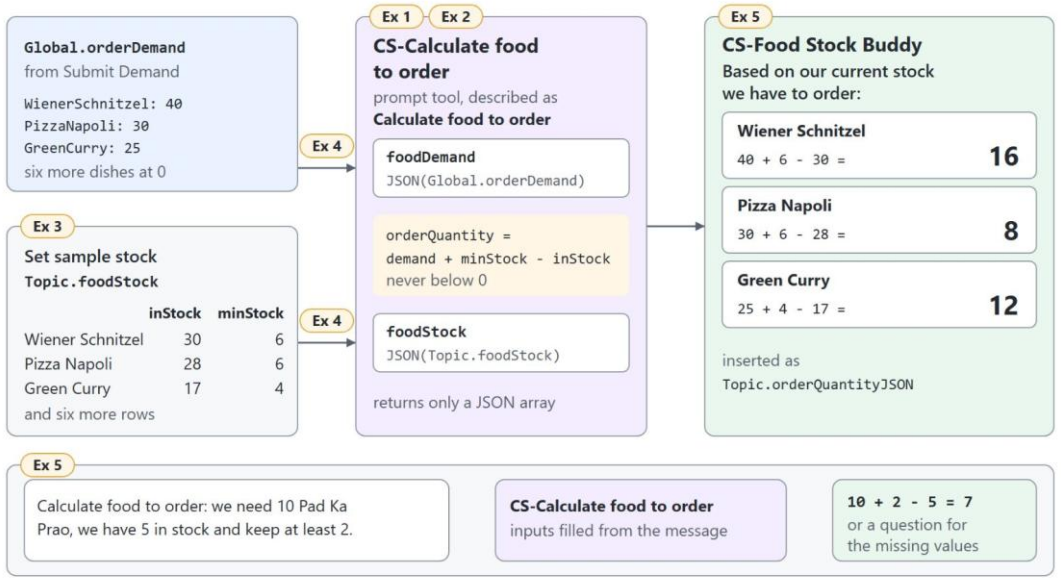
A fixed instruction for the canteen order

Calculating the canteen order with a prompt

CS-Food Stock Buddy passes demand and stock as JSON

- The topic binds both inputs with **JSON()** and reads the answer from the output's `text` field
- kind: InvokeAIBuilderModelAction

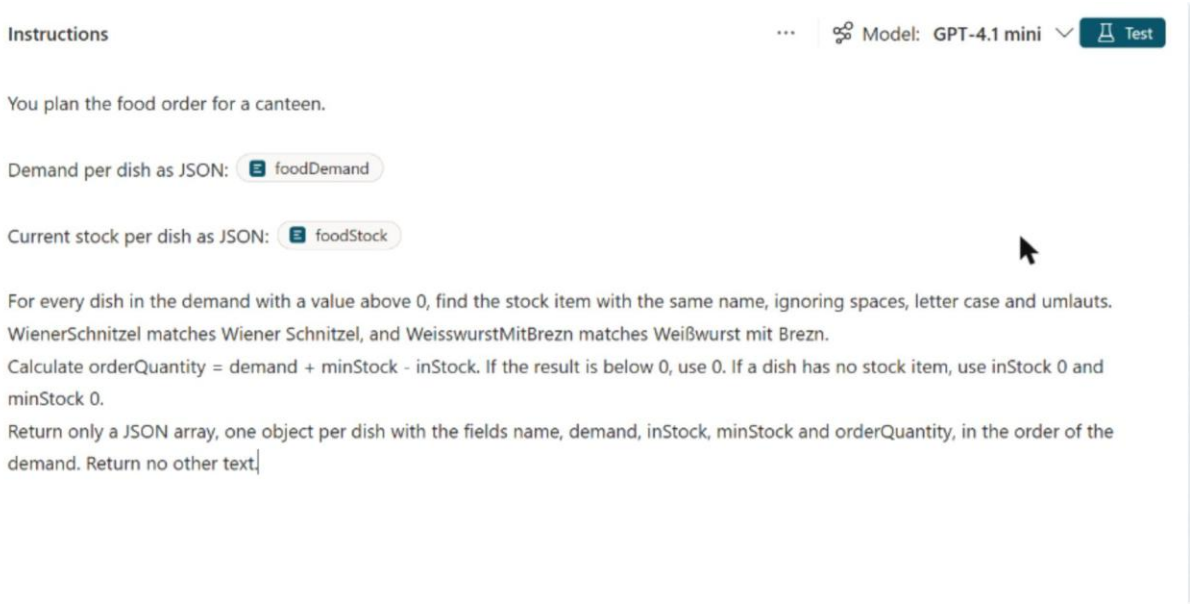
```
- kind: InvokeAIBuilderModelAction
  input:
    binding:
      foodDemand: =JSON(Global.orderDemand)
      foodStock: =JSON(Topic.foodStock)
  output:
    binding:
      predictionOutput:
Topic.orderQuantityRecord
```



Writing and testing a prompt

Instructions, typed inputs and a test before anyone uses it

- The **instructions** say what the model does and what the answer looks like
- Typing **/** inserts an input; foodDemand and foodStock are Text inputs
- **Test** with sample values first, and save only when the output has the right shape
- "Return only a JSON array" and **temperature 0** keep the output easy to parse



02

Dataverse: Knowledge & Business Records

Travel claims saved as table rows

Travel claims as Dataverse rows

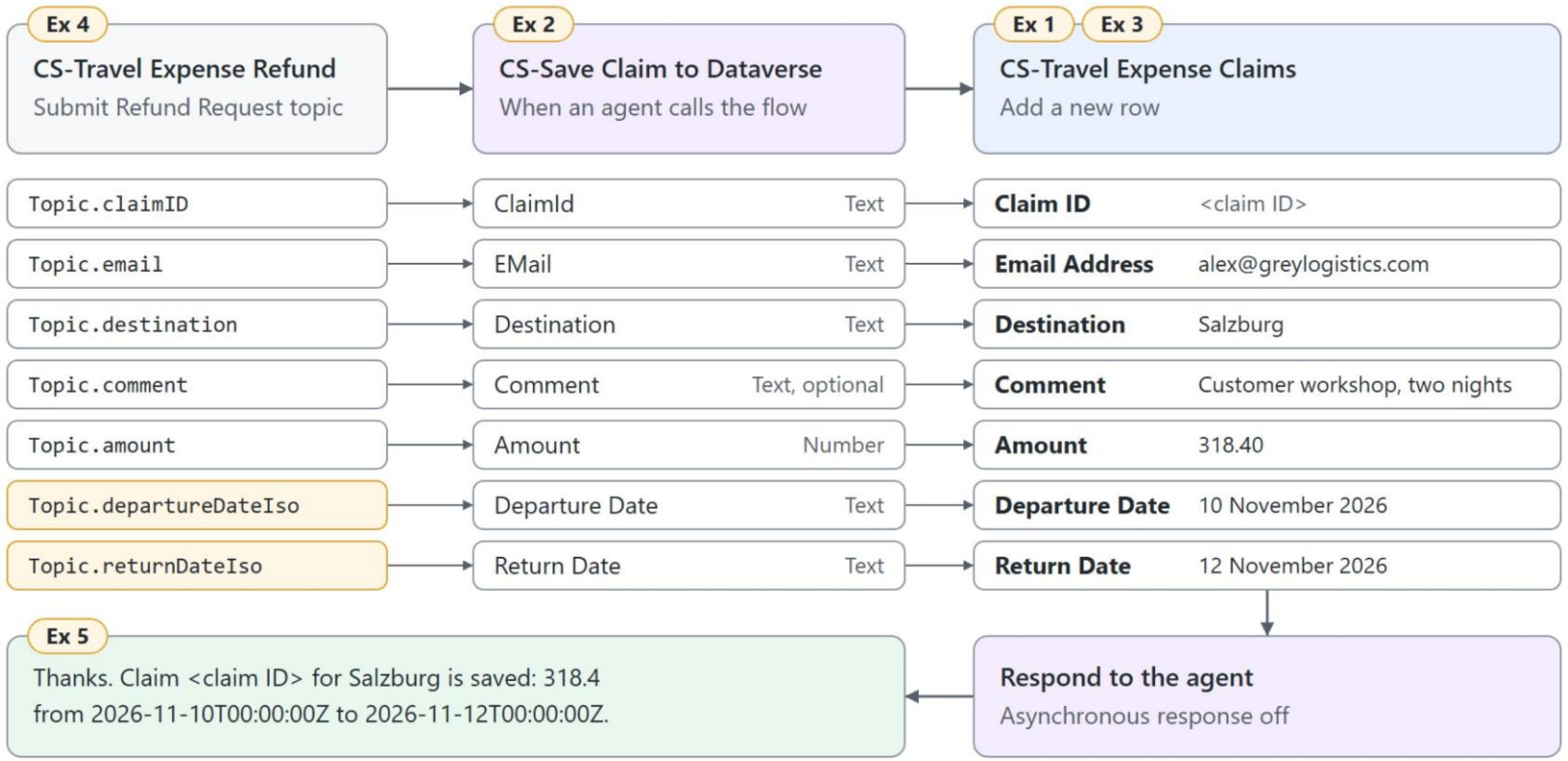
Business records the agent reads and writes

A claim that stays in the chat can't be approved, paid or reported on, so **CS-Travel Expense Refund** writes each one as a row in a Dataverse table



Saving a claim through an agent flow

Seven topic variables, seven flow inputs, one new row

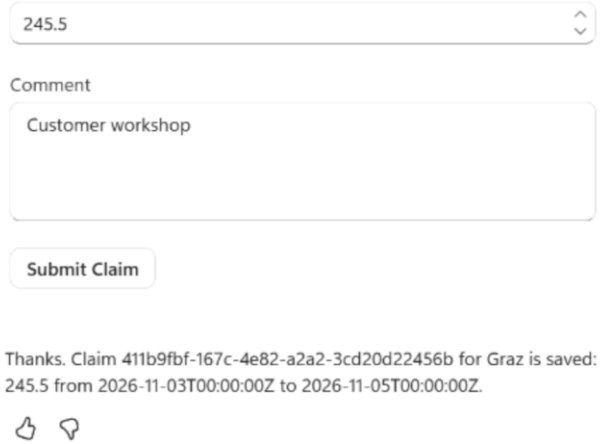


The **Action** node passes the claim, ISO dates included, to CS-Save Claim to Dataverse, which adds the row and responds to the agent

The saved claim in the test chat

The confirmation carries the claim ID and ISO dates

- After **Submit Claim** the agent confirms the claim with its ID, amount and dates
- The dates arrive as **ISO timestamps**, the format the Date and time columns accept
- The proof is the **row in Power Apps**, matched against what you typed
- Writes can also run through a **Dataverse connector tool** that asks before running



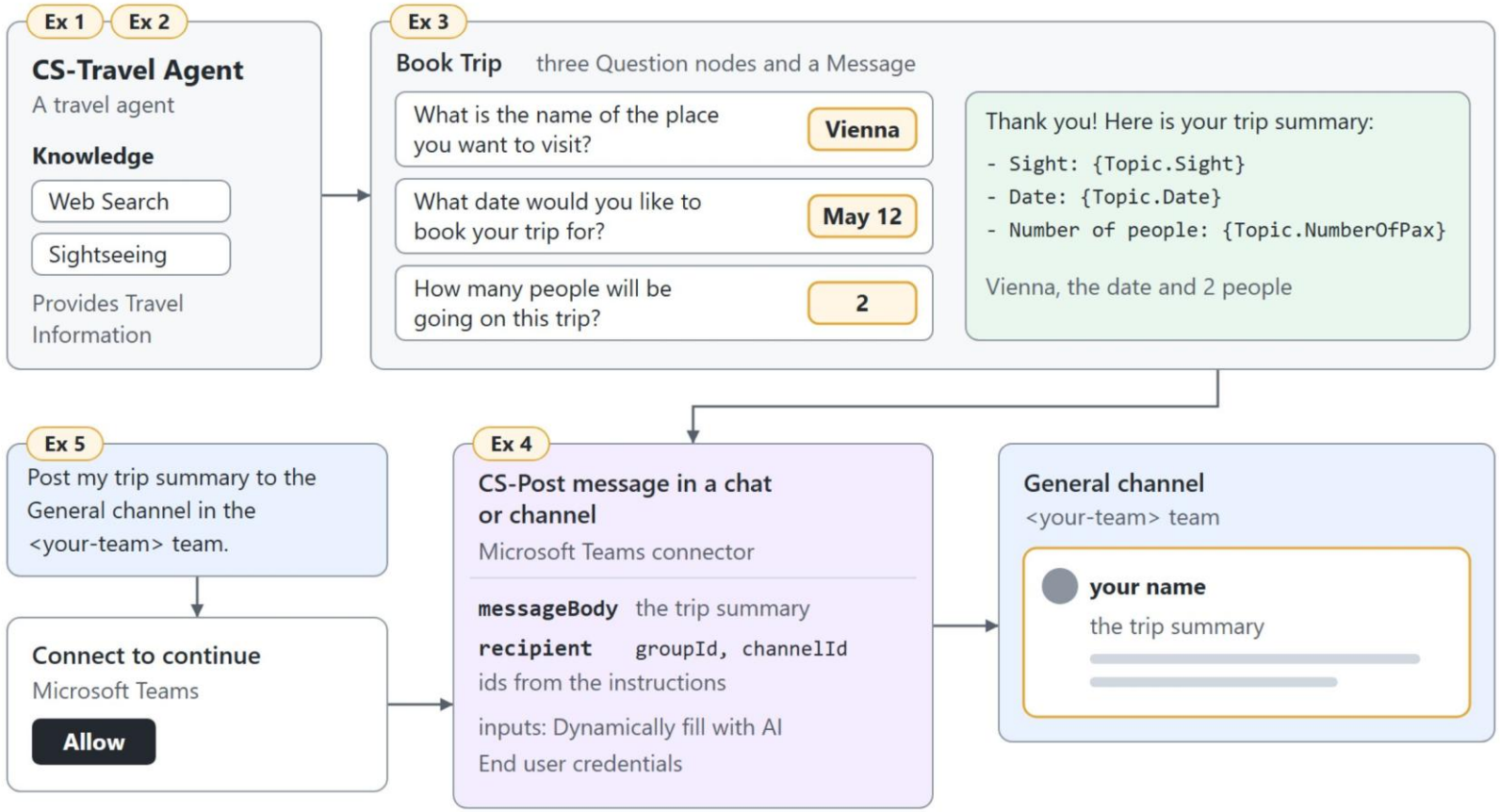
03

Connector Tools: Prebuilt Connectors

One Teams action, filled from the chat

Posting a trip summary to Teams

One Microsoft Teams action as a tool of CS-Travel Agent

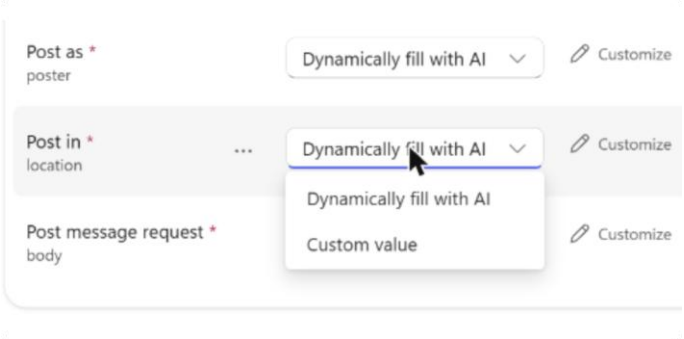


Book Trip collects Vienna, May 12 and 2 people, then **CS-Post message in a chat or channel** posts the summary to the General channel

How the agent fills tool inputs

Dynamically fill with AI or a custom value, per input

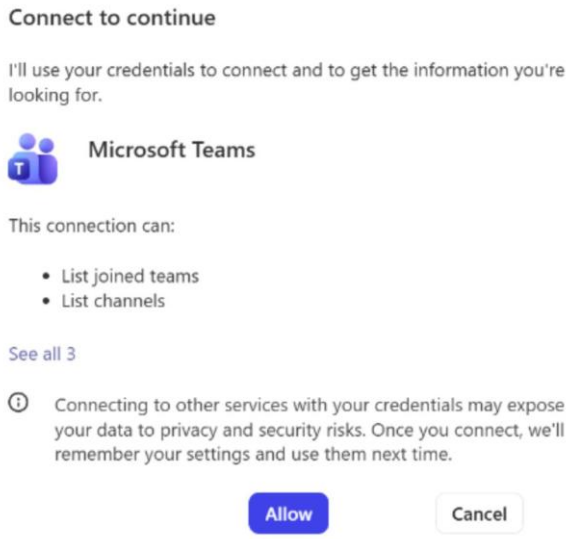
- **Dynamically fill with AI** (the default) takes the value from the conversation or asks for it
- A **custom value** is fixed: a value, a variable or a formula such as `System.User.Email`
- The Teams tool keeps Post as, Post in and Post message request on dynamic filling
- The orchestrator chooses the tool by its **name and description**, so rename it after Add and configure



Whose credentials a tool uses

End user credentials ask each person to connect once

- **End user credentials** (the default): each person connects on first use and acts with their own access
- **Maker-provided credentials** run every call with the author's access
- Maker-provided credentials need an **authenticated channel**, and admins can turn them off
- The Publish dialog then warns: "Your agent runs tools that use the author's credentials."



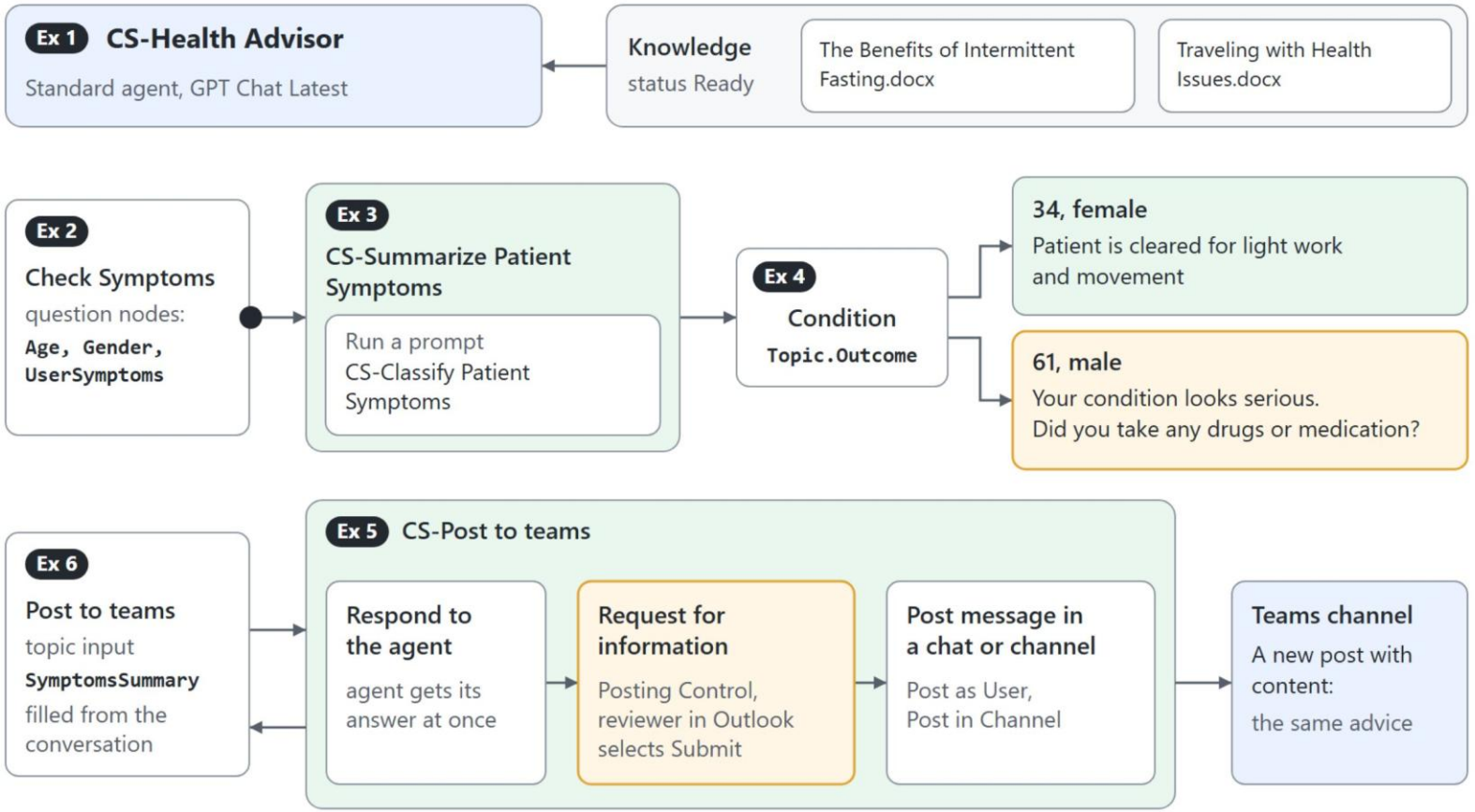
04

Agent Flows: Deterministic Automation

Fixed steps with a prompt and a reviewer

The Health Advisor's two agent flows

A prompt step classifies, a review step guards the post

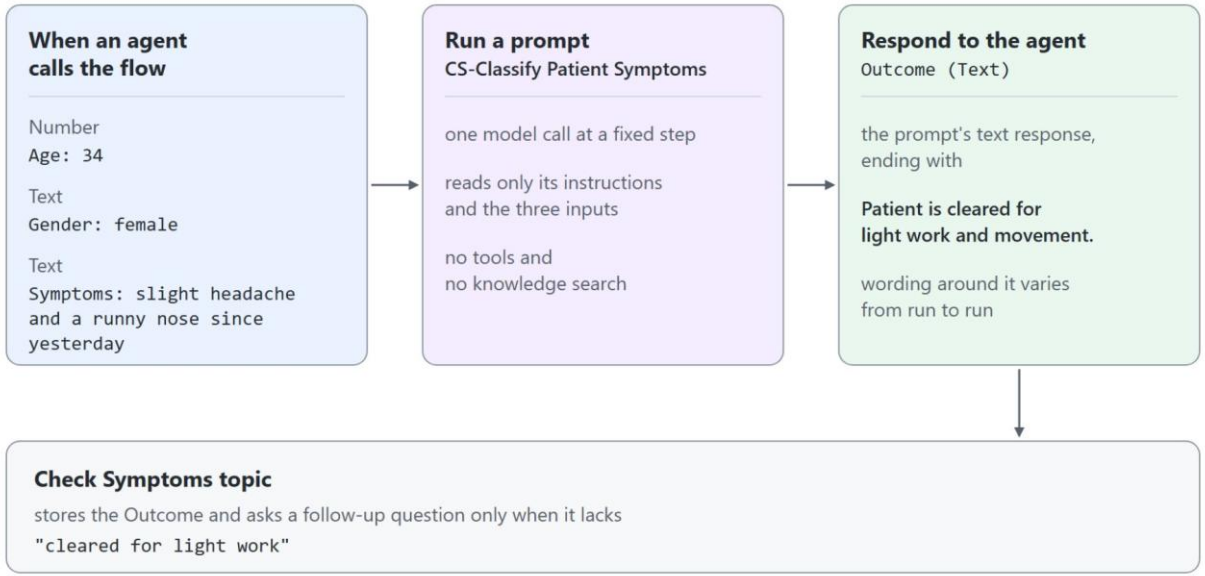


- **Check Symptoms** branches on the Outcome of CS-Summarize Patient Symptoms, and **CS-Post to teams** answers the agent at once, then waits for a reviewer

A prompt step inside an agent flow

CS-Classify Patient Symptoms classifies, the topic branches

- **Run a prompt** makes one model call at a fixed point in the flow
- The prompt reads only its **instructions** and the age, gender and symptoms it is given
- With 34, female and mild symptoms, the answer ends "Patient is cleared for light work and movement."
- **Respond to the agent** returns that text as Outcome, and the topic branches on it



A reviewer before the Teams post

The agent gets its answer first, the post waits for a person

- **Respond to the agent** sits right below the trigger, so the chat gets its answer at once
- **Request for information** emails the reviewer a Posting Control request through Outlook
- The flow posts to the Teams channel only after the reviewer selects **Submit**
- An agent waits at most 100 seconds for a flow; the review step runs on after the response



05

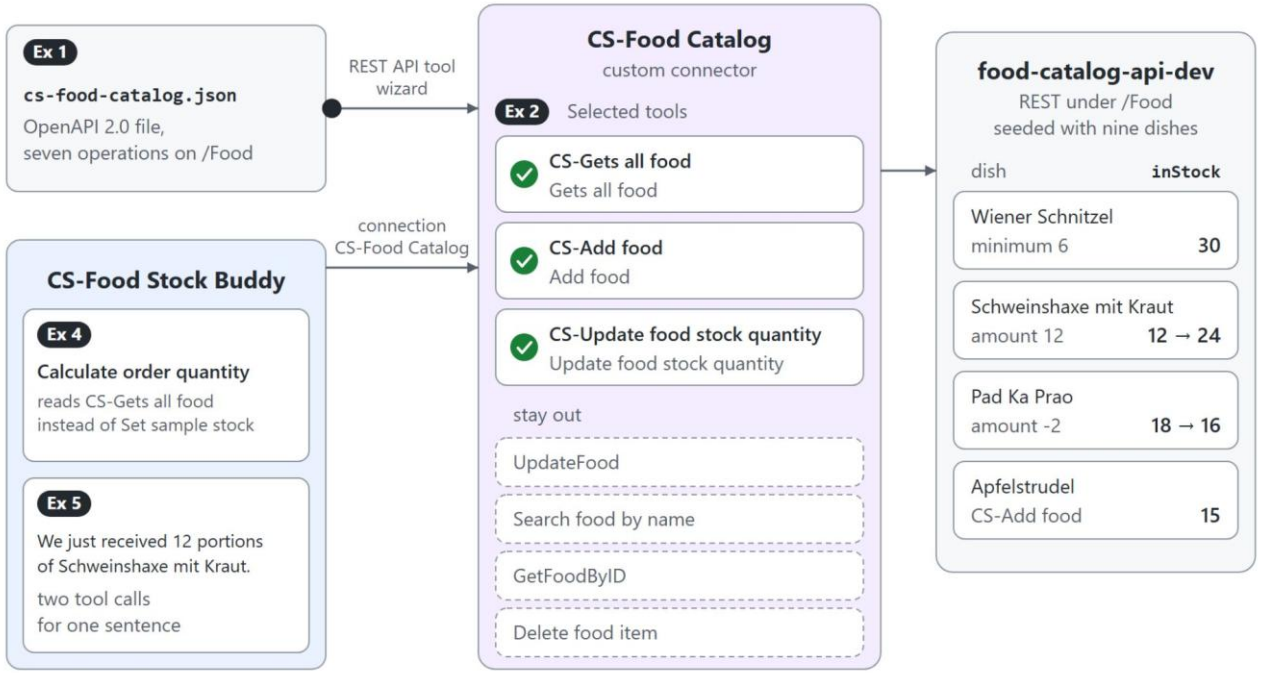
Custom Connectors & REST APIs

Live canteen stock from an OpenAPI file

Live stock from the food catalog API

An OpenAPI file becomes CS-Food Catalog and three tools

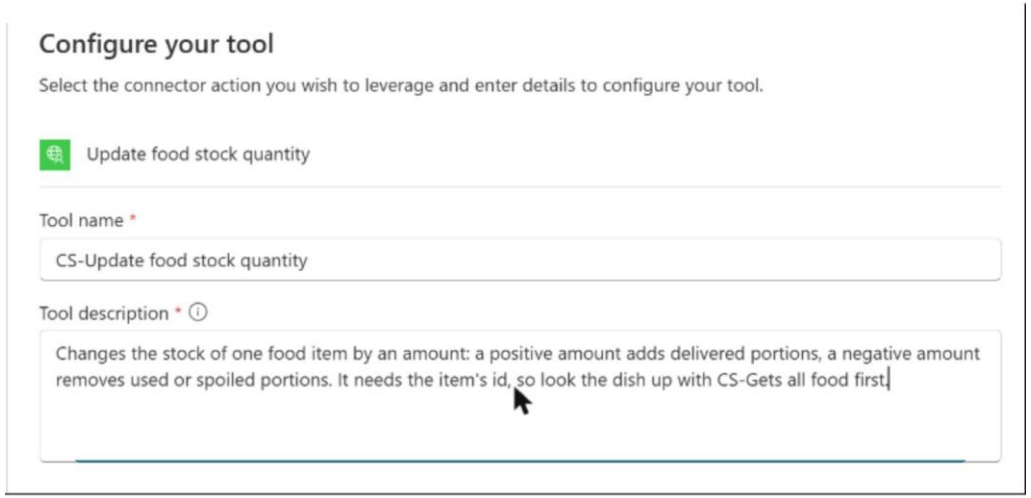
- The **REST API tool** wizard turns cs-food-catalog.json into the CS-Food Catalog custom connector
- Three of seven operations become tools: CS-Gets all food, CS-Add food and CS-Update food stock quantity
- UpdateFood, the two lookups and **Delete food item** stay out
- Calculate order quantity now reads **CS-Gets all food** instead of Set sample stock



Tool descriptions the orchestrator routes on

A description can also set the call order

- Each description names the **job**, the data and when to use the tool
- "Look the dish up with CS-Gets all food first" sets the **call order**
- Without that line the agent guessed an id and got **Not Found**
- The wizard asks for a description of **every input and output** before Next turns on



Booking a delivery in one sentence

The orchestrator chains a lookup and an update

- "We just received 12 portions of Schweinshaxe mit Kraut" makes the agent **look up the dish, then add 12 to its stock**: 24 portions on the seeded database



06

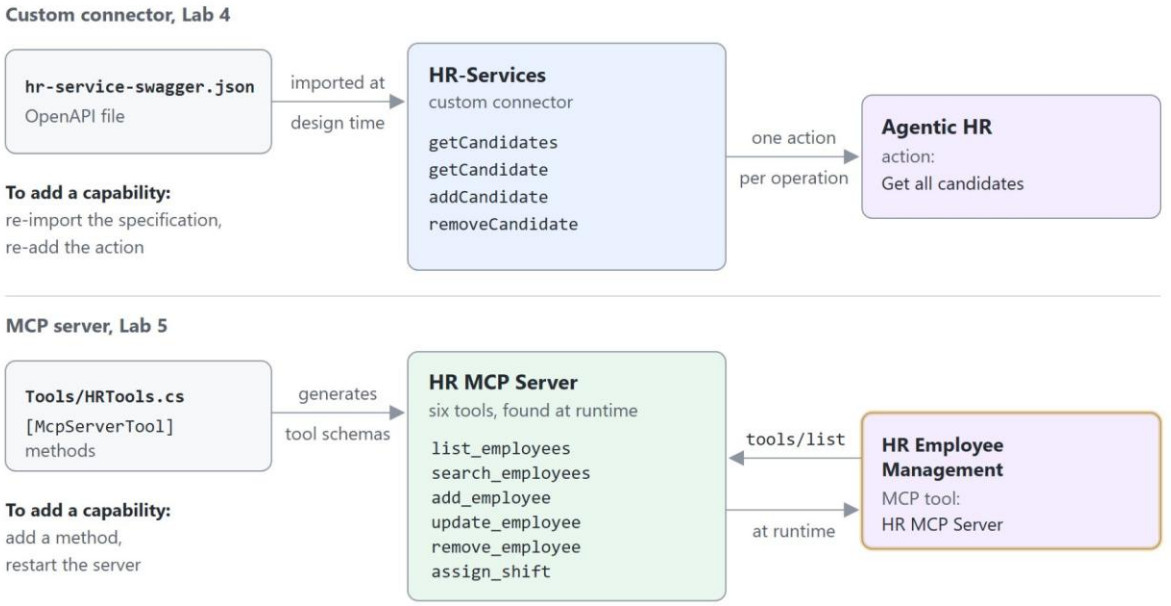
MCP Servers as Agent Tools

Staff and shifts discovered at runtime

Tool discovery with MCP

A connector is fixed at import, an MCP server lists its tools at runtime

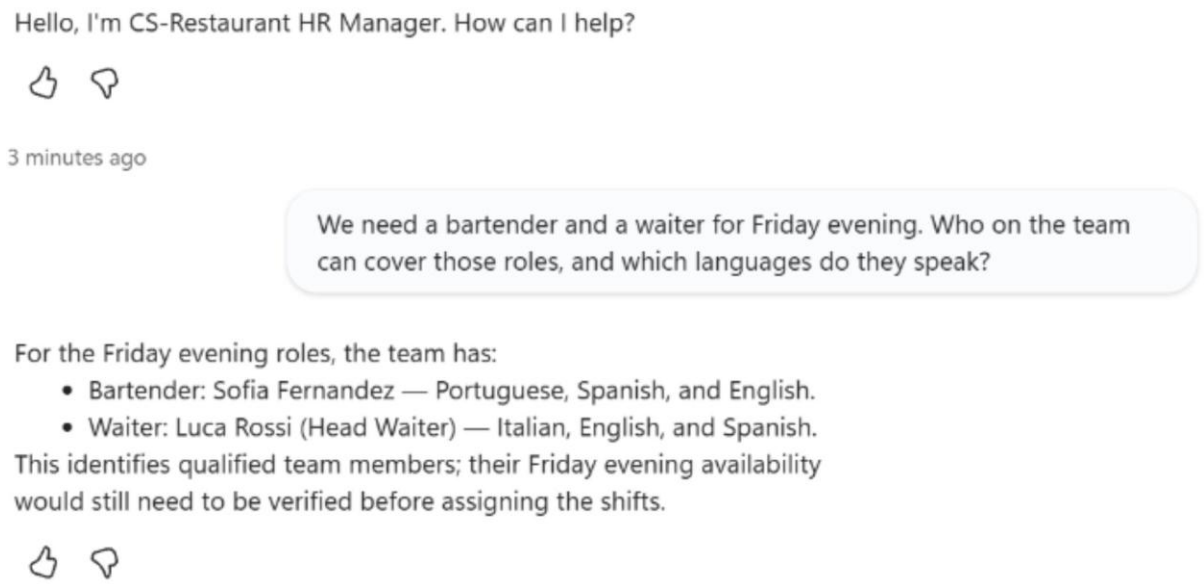
- A **custom connector** keeps the operations of the OpenAPI file you imported
- An **MCP server** answers `tools/list` at runtime with its current tools
- Tool descriptions come from the server's `[Description]` attributes, so a vague one is fixed there
- **Adding a tool** means adding a method and restarting the server
- MCP needs **generative orchestration**, and topics cannot call a server directly



Finding staff through the HR MCP server

CS-Restaurant HR Manager reads the team before it plans

- The question names no tool; the orchestrator picks **CS-HR MCP Server** from its description
- The answer names **Sofia Fernandez** and **Luca Rossi** with their languages, from the seeded staff
- The server was connected through the **MCP onboarding wizard**: name, description, URL, no authentication
- A reply that names people missing from the server's data means **no tool ran**



Booking shifts after a confirmation

The agent reads freely and writes after a yes

- The agent books **Sofia on the bar and Luca on waiter duty** for 2026-11-06 only after you confirm, and the server sets each shift to eight hours



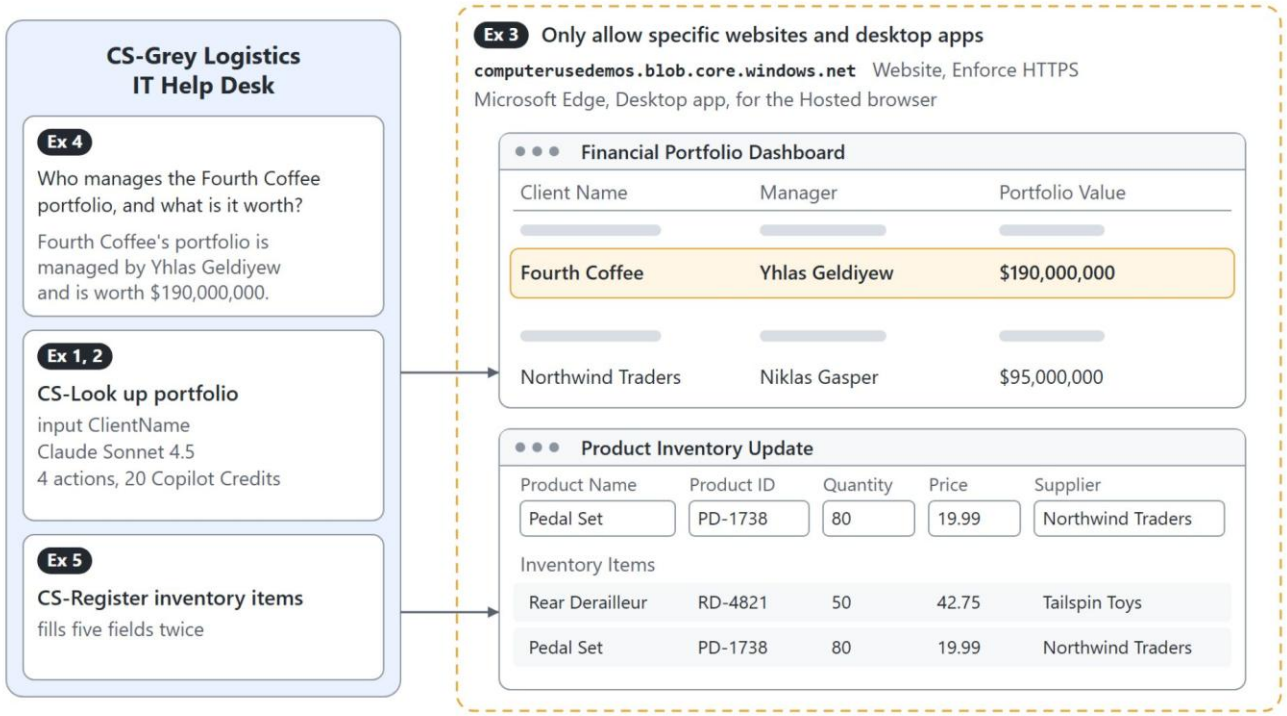
07

Computer Use: Apps Without APIs

Reading and filling web apps by screen

Computer use on web apps without an API

Two tools for the help desk agent, fenced to one site

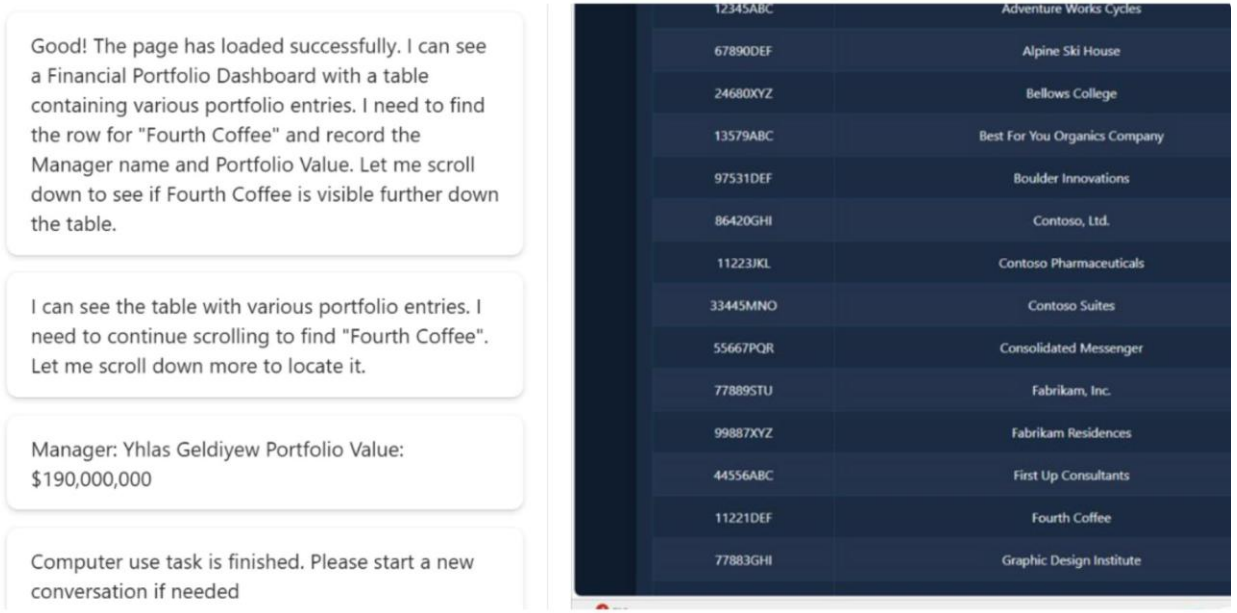


- **CS-Look up portfolio** reads a client's manager and value, **CS-Register inventory items** types two products into a form, both limited to one HTTPS site

A computer use test run

The model's reasoning next to the hosted browser

- The left column shows the model's **reasoning** after each step of the run
- The hosted browser scrolls the Financial Portfolio Dashboard down to **Fourth Coffee**
- Claude Sonnet 4.5 returned **Yhlas Geldiyew, $190,000,000** exactly as the page shows it
- In chat the same lookup took **4 actions**: 20 Copilot Credits at 5 per step



Takeaways

Pick the tool by the job

- A **prompt** for the canteen order, an **agent flow** for fixed steps, a connector or REST API for one service
- **MCP** for a toolset many agents share, **computer use** only where no API exists
- The orchestrator chooses by **name and description**, so write them in your users' words
- Rows, posts, shifts and mails go out **after a confirmation**, with credentials you chose on purpose
- The **activity map** shows which tool ran and what it was given

